Generating Music with Machine Learning

UE24CS352A Machine Learning mini-project - Team C-14 - Krrish Singla and Kasissnu Ssinha

1. Problem and objective

This project investigates whether a machine-learning model can learn local musical structure from piano MIDI performances and generate new music. A MIDI performance is converted into a sequence of note-on, note-off, and time-shift tokens. A model predicts the next token repeatedly, and the generated sequence is converted back into a MIDI file. We compare a simple order-2 Markov model with an LSTM neural sequence model.

2. Dataset and representation

We use classical piano performances from the MAESTRO MIDI dataset. Files are split at the piece level into training, validation, and test sets. The final processed data contains 3,240,519 training tokens, 386,786 validation tokens, and 363,359 test tokens.

The vocabulary contains 356 events: 128 note-on tokens, 128 note-off tokens, and 100 time-shift tokens representing 10-1000 ms intervals. This representation preserves pitch and timing, and the decoder converts tokens back into standard MIDI files.

3. Models and experiment

The baseline is an order-2 Markov model. It estimates the next-token distribution from the preceding two tokens, backs off to shorter contexts when necessary, and uses smoothed unigram probabilities for unseen contexts.

The main model is a PyTorch LSTM with an embedding layer, two recurrent layers, dropout, and a linear output layer over the vocabulary. It is trained with cross-entropy loss and Adam optimization. Both models use the same token representation and data split. The final LSTM run used 8,000 training steps on CPU.

4. Results

Model	Test loss	Perplexity	Accuracy
Unigram (no context)	4.4810	88.32	11.89%
Markov (order 2)	3.8966	49.23	18.58%
LSTM (2 layers, 256 hidden, 8000 steps)	2.3982	11.00	32.17%

All models are scored on the same 361,845 target tokens: the held-out test pieces (363,359 tokens) are cut into 1,419 non-overlapping windows of 256 tokens, and every token after the first in a window is a prediction target. The LSTM sees the window so far, the Markov model its last two tokens. The LSTM performs better on every metric. Its lower perplexity means it assigns higher probability to held-out token sequences. The Markov model is simple and interpretable, but exact short contexts are sparse in polyphonic music; the LSTM can generalize beyond an exact two-token lookup.

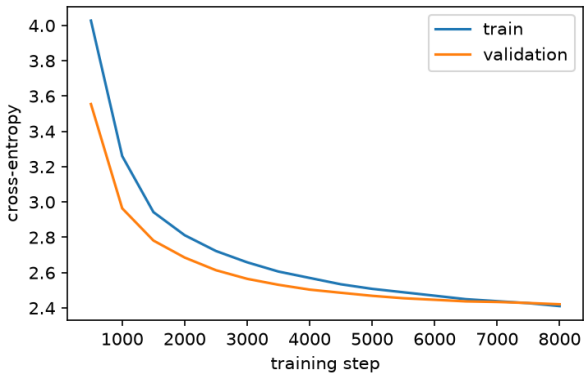

Figure 1. LSTM training and validation cross-entropy.

5. Generation and evaluation

Both models generate new token sequences that are decoded into MIDI files. Samples are written to outputs/midi/ by python -m src.generate, and the repository includes a Streamlit application for selecting a model, controlling generation length and temperature, listening to synthesized audio, viewing a piano roll, and downloading the MIDI result.

The numerical evaluation measures next-token prediction, while music quality is not fully captured by loss or accuracy. A blind listening survey (six equal-length clips: two human, two Markov, two LSTM) was designed, but no responses are reported here.

6. Conclusion

The completed experiment shows that the LSTM is a stronger next-token predictor than the order-2 Markov baseline on this dataset: accuracy improves from 18.58% to 32.17%, and perplexity decreases from 49.23 to 11.00. This improvement does not automatically prove that the LSTM creates better music, because musical quality also depends on repetition, harmony, phrasing, dynamics, and listener preference.

7. Limitations and future work

The current representation models pitch, note-off events, and timing, but does not explicitly model expressive velocity, key, meter, or rich multi-instrument structure. The generated audio uses a simple piano-like synthesizer. Future work could add velocity and chord features, and conduct a larger listener study. A Transformer could also be explored after the two-model comparison is complete.

8. Reproducibility

Install dependencies with pip install -r requirements.txt. Run python -m src.preprocess --raw data/raw/maestro --max-files 200, then python -m src.markov --order 2 and python -m src.train_lstm --steps 8000 (torch is needed only for training; evaluation, generation and the demo run from the exported numpy weights in models/lstm.npz). Reproduce the table with python -m src.evaluate, generate samples with python -m src.generate, and launch the demonstration with streamlit run app.py.

Project repository structure and experiment details are documented in README.md.